

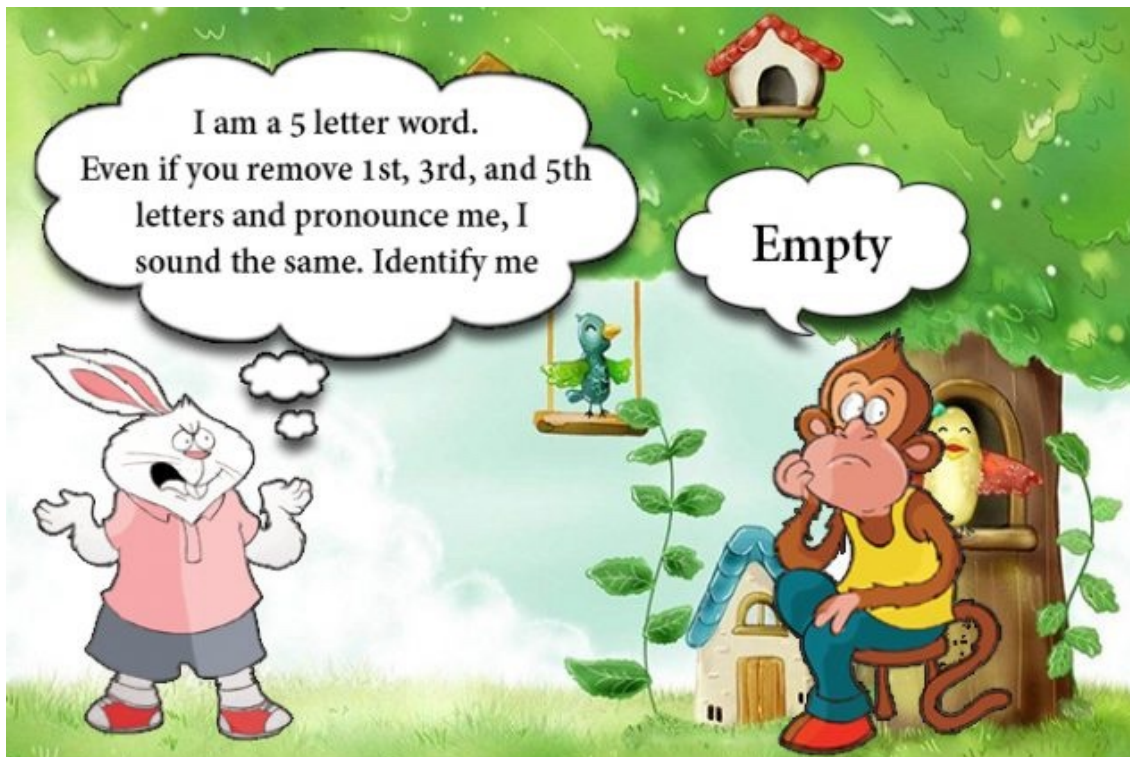
Stacks 1

TABLE OF CONTENTS

1. Stacks
2. Operations in Stacks
3. Implementing Stacks using Arrays
4. Implementing Stacks using Linked List
5. Double Character Trouble



Notes

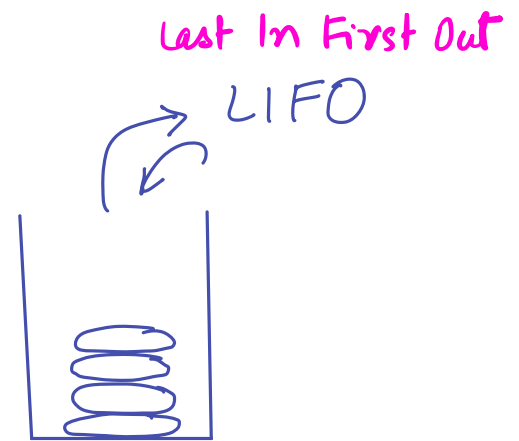
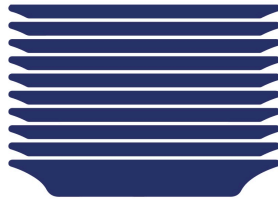




Stacks

A stack is a linear data structure that store info in a sequence, from bottom to top.

1. Stack of Plates



2. Stack of Chairs



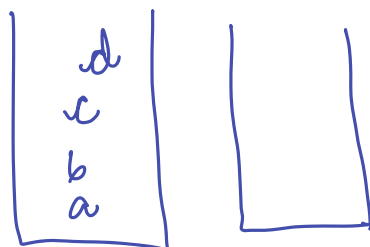


3. Stack of Functions

4. Browser History

Today		
☐	12:56	Trees 5 notability.com
☐	12:40	Trees 4 notability.com
☐	09:52	Trees - 3 notability.com
☐	09:52	Google Sheets: Sign-in docs.google.com

* Undo/ Redo.





Operations in Stack

1. Push (x) - insert the element from top of stack
2. Pop () - remove the element from top of stack
3. Top () / Peek () - check element on top
4. isEmpty () - if the stack is empty, return true, else return false

$O(1)$

push(2) ✓

push(4) ✓

push(7) ✓

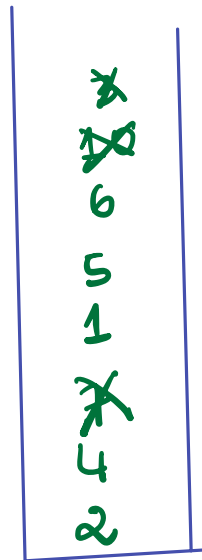
pop() ✓

push(1) ✓

isEmpty() → false

push(5) ✓

push(6) ✓



top() → 6

push(10) ✓

push(3) ✓

pop() ✓

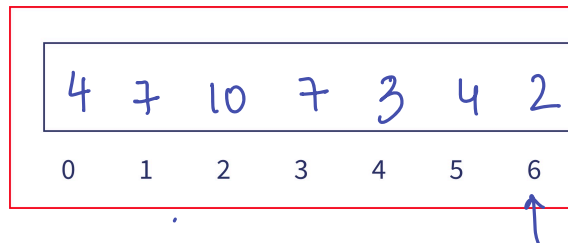
pop() ✓

top() → 6

isEmpty() → false

Implementing Stacks using Arrays A[]

N (capacity) = 7



top = ~~-1~~ ~~0~~ ~~1~~
~~2~~ ~~3~~
2

push(4)
push(7)
push(6)
pop()
top() → 7
push(10)
push(9)
pop

O(1)
void push(int n) {
 if (top == N-1) {
 print "Stack Overflow"
 return
 }
 top++;
 A[top] = n;
}

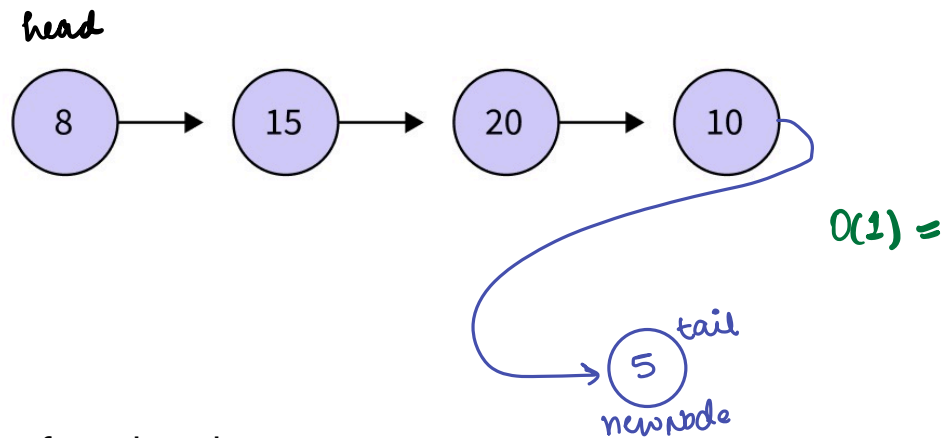
O(1)
void pop() {
 if (top == -1)
 return
 top--
}

O(1)
int top() {
 if (top == -1)
 return -1
 return A[top]
}

O(1)
bool isEmpty() {
 if (top == -1) {
 return true
 }
 else return false
}



Implementing Stacks using Linked List

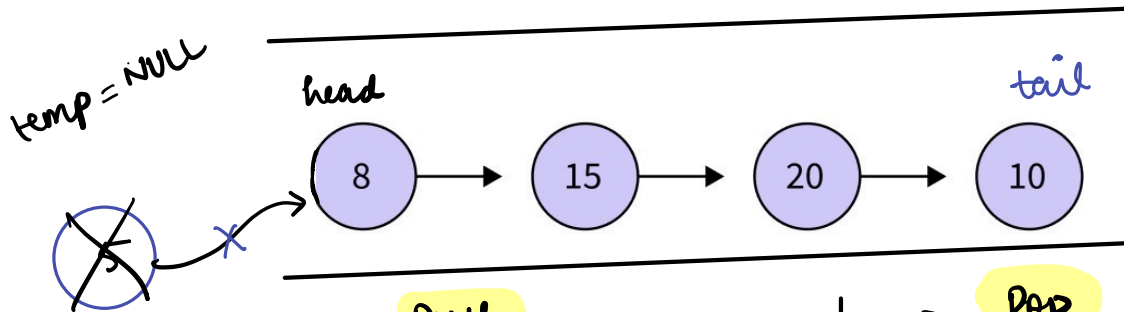


A. Push at head, Pop from head.

~~B. Push at tail, Pop from tail. $O(1)$ $O(N)$ xx~~

~~C. Push at head, Pop from tail. xx~~

A. Push at head, pop at head



If LL was missed, watch session → classes & obj.

Push
 $O(1)$ {
 newNode.next = head
 head = newNode
}

Pop
 $O(1)$ {
 temp = head
 head = head.next
 free(temp) // C++
 temp.next = NULL
 temp = NULL
}

isEmpty()

if (head == NULL)
 return true
 else return false

top()

if (head == NULL) {
 return -1
 }
 else { return head.val }

1) No overflow in LL
 2) Underflows can be there.



```
Stack <int> s;
Stack <Integer> s = new Stack <>();
```

Q. Implement Stack using queue

Stack $\rightarrow$ LIFO
Queue $\rightarrow$ FIFO

2 ways to do

$\hookrightarrow$ while inserting handle the case where last becomes first in queue

$\hookrightarrow$ while popping handle the case to get last $\Rightarrow$ brute using 2 queue

using case 1 $\rightarrow$ handling addition in a way that last addition becomes first

```
push(x) {
    q1.push(x)
```

```
    for(i=0; i < q1.size; i++) {
        q1.add(q1.pop());
```

```
    }
```

```
}
```

```
pop() {
    return q1;
```

q1 $\rightarrow$ [4|3|2|1]

push(7)

↓

0	1	2	3	4
4	3	2	1	7

 size = 4
 send all these back

[7|4|3|2|1]

7
4
3
2
1

< Question > : Check whether the given sequence of parenthesis [], {}, () is valid or not?

Example : () [{}] () ✓

() [] ✓

{} } ✓

{ } X

[{] X

([{}][]()) ✓

→ when will it come as valid ?

↳ for each closing bracket the last opening bracket was the pair of the current closing bracket exist or stack is not empty



If entire expression is valid & its sub-expressions are also valid. This follows a recursive structure

What if whenever we encounter a valid pattern, we remove it?



Example : $((\{\})\{ })$



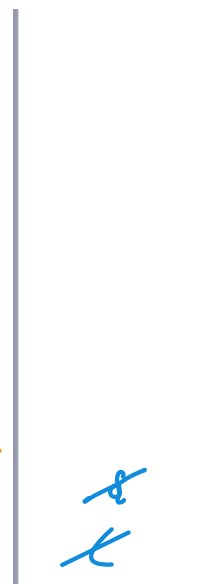
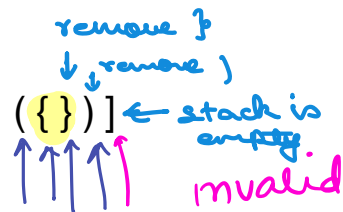
Invalid

invalid because
① last element was not
matching pair



② no opening was
present a closing bracket
aka extra closing was
present

Example : $((\{\})\{\})$

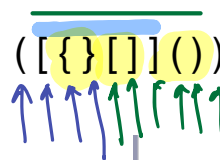


Example : $[()]$

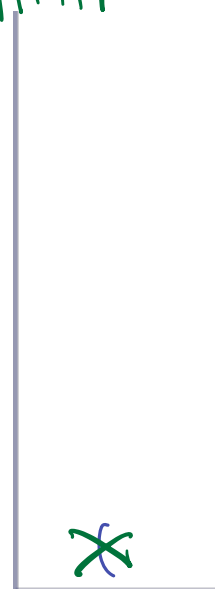


Invalid

③ after all the
iteration stack
was not empty
aka extra opening
was present



valid $\checkmark$





pseudocode

Traverse the string

if an open bracket
push on stack

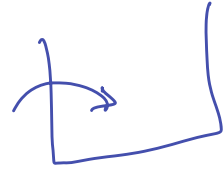
else if stack is not empty & a close bracket & is of same type
pop from stack

else return false

if (stack is empty)

return true

else return false



TC

$O(N)$

SC

$O(N)$

([

[
(



```

public class Solution {
    public int solve(String A) {
        HashMap < Character, Character > mp = new HashMap < Character, Character > ();
        Stack < Character > st = new Stack < Character > ();
        mp.put(')', '(');
        mp.put('}', '{');
        mp.put(']', '[');
        for (int i = 0; i < A.length(); i++) {
            char c = A.charAt(i);
            if (c == '(' || c == '[' || c == '{') {
                // push any opening bracket into the stack
                st.push(c);
            } else if (st.empty() || st.peek() != mp.get(c)) {
                // check if the last unpaired opening bracket is of the same type
                // as the current closing bracket
                return 0;
            } else {
                st.pop();
            }
        }
        // checks if all the opening brackets are paired
        if (st.empty())
            return 1;
        return 0;
    }
}

```

→ to store all pairs
map

)	→	(
}	→	{
]	→	[

→ last of stack is not matching top

Trial

- ① use a map to store all pairs
- ② loop thru string & check
 - ↳ if open add it to stack
 - ↳ if closing then check
 - ↳ stack is not empty & last one matches to opening

Charade, Charade
HashMap < > map = new HashMap < Char, Char > ();

map.put (' (' , ') ')

map.put (' [' , '] ')

map.put (' { ' , ' } ')

for (i = 0 ; i < A.length() ; i++) {

char ch = A.charAt (i) ;

if (ch == ' (' || ch == ' [' || ch == ' { ') {

stack.push (ch)

} else if (!stack.isEmpty() && stack.peek() ==
map.get (ch)) {

→ closing found & valid

stack.pop()

} else {

return 0

}

}

if (!stack.isEmpty()) {

return 0

}

return 1



Double Character Trouble

Given a string `str`. Remove equal pair of consecutive elements till possible.

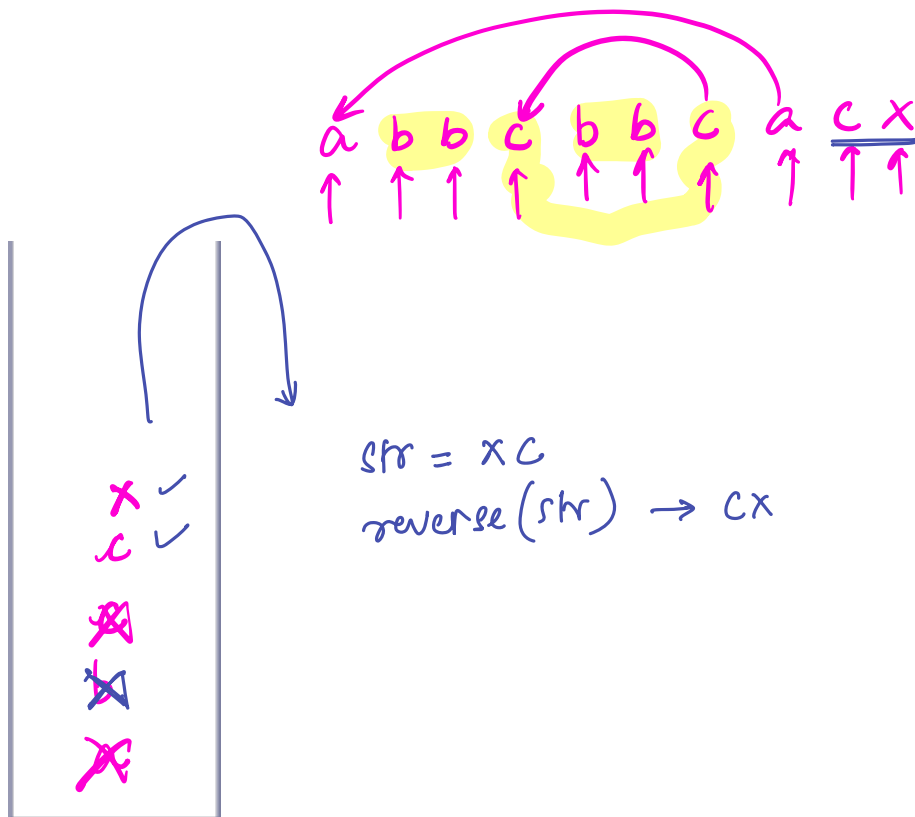
Return the strings without adjacent duplicates.

1. $a \cancel{b} \cancel{b} d \rightarrow ad$ *both b will be removed*

2. $a \cancel{b} \cancel{c} \cancel{c} b d e \rightarrow a \cancel{b} \cancel{b} d e \rightarrow ade$

3. $a \cancel{b} \cancel{b} b d \rightarrow abd$

4. $a \cancel{b} \cancel{b} c \cancel{b} \cancel{b} c a c x \rightarrow a \cancel{c} \cancel{c} a c x \rightarrow \cancel{a} \cancel{a} c x \rightarrow cx$





(1) Traverse the string

```
if (A[i] == st.top()) {  
    st.pop()  
}  
else {  
    st.push(A[i])  
}
```

TC $\rightarrow O(N)$
SC $\rightarrow O(N)$

(2) Take chars from stack in a string from top to bottom.

(3) Reverse & return

Q Implement Min Stack (Striver)

↳ what is a min stack?

↳ normal stack but it has an extra get min
fun that returns min

T.C $\rightarrow O(1)$

Space $\rightarrow O(2 \times n)$

↓
as we are
storing a pair

2 ways $\rightarrow$ ① have a stack with Pairs

Stack < Pair > stack = new Stack < > ();

push (n) {

if (stack.isEmpty()) {

stack.push (new Pair (n, n));

} else {

stack.push (new Pair (n,
Math.min (n,
stack.peek().
min
))

}

pop () {

return stack.pop();

}

each pair

↳ represent
curr element

↳ & the min
element it has
found till now

Optimised : how to reduce Space complexity
from $O(2n) \rightarrow O(n)$

how about lets keep a min variable

push (12) .
push (15) .
push (10) .
pop ()
get min

min $\rightarrow$ ~~12~~ 10
stack $\rightarrow$ 12, 15, ~~10~~

I dont have a way to back trace to 12 if 10 is popped out

So instead of storing val directly why not store
 $\rightarrow 2 \times \text{val} - \text{prev Min} = \text{new Stack Value}$
cohenever we see a value smaller than min

So

push (12) -
push (15) .
push (10) .
pop ()
get min

min $\rightarrow$ ~~12~~ 10
stack $\rightarrow$ 12, 15, 8
 $\downarrow$
stack empty

at val 10 is < 12 (curr min)
so store $\rightarrow 2 \times 10 - 12$
8

Push

```
if (st.isEmpty()) {
    min = val
    st.push(val)
} else {
    if (val < min) {
        st.push(2 * val - min)
        min = val
    } else {
        st.push(val)
    }
}
```

Pop

```
if (st.isEmpty()) {
    return
}
val = st.pop()
if (val < min) {
    min = 2 * min - val
}
```

Q

Expression Evaluation

Infix Expression

→ Operator comes in middle

8 * 5 + 4

operand operator operand
↓ ↓ ↓
8 * 5

C) ^ → 3
* / → 2
+ - → 1

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to and not equal to	left to right
&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
	Logical OR	left to right
?:	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	Comma operator	left to right

when 2 have same weightage use the one which comes first

Steps

① Compare betⁿ 2 operators

So $A+B \rightarrow$ is not a comparison

$AB+$

$A+B - C+B$ Here 3 are comparison

$AB+ C- B+$
same

$AB+ C- B+$

Prefix : operator comes first then the
operands $\rightarrow +AB$
 $\rightarrow$ used in LISP language

Post fix : operator comes after operands $\rightarrow AB+$
 $\rightarrow$ used in stacks based calculator

Rules to convert infix to Postfix

- ① know priority $\rightarrow$ chart above
- ② no 2 operator of same priority can stay together
- ③ lowest priority cannot be placed after highest priority

Infix	Postfix	Prefix
$A + B$	$AB+$	$+AB$
$A - B$	$AB-$	$-AB$
$A * B$	$AB*$	$*AB$
A / B	$AB/$	$/AB$

Infix to postfix : striver

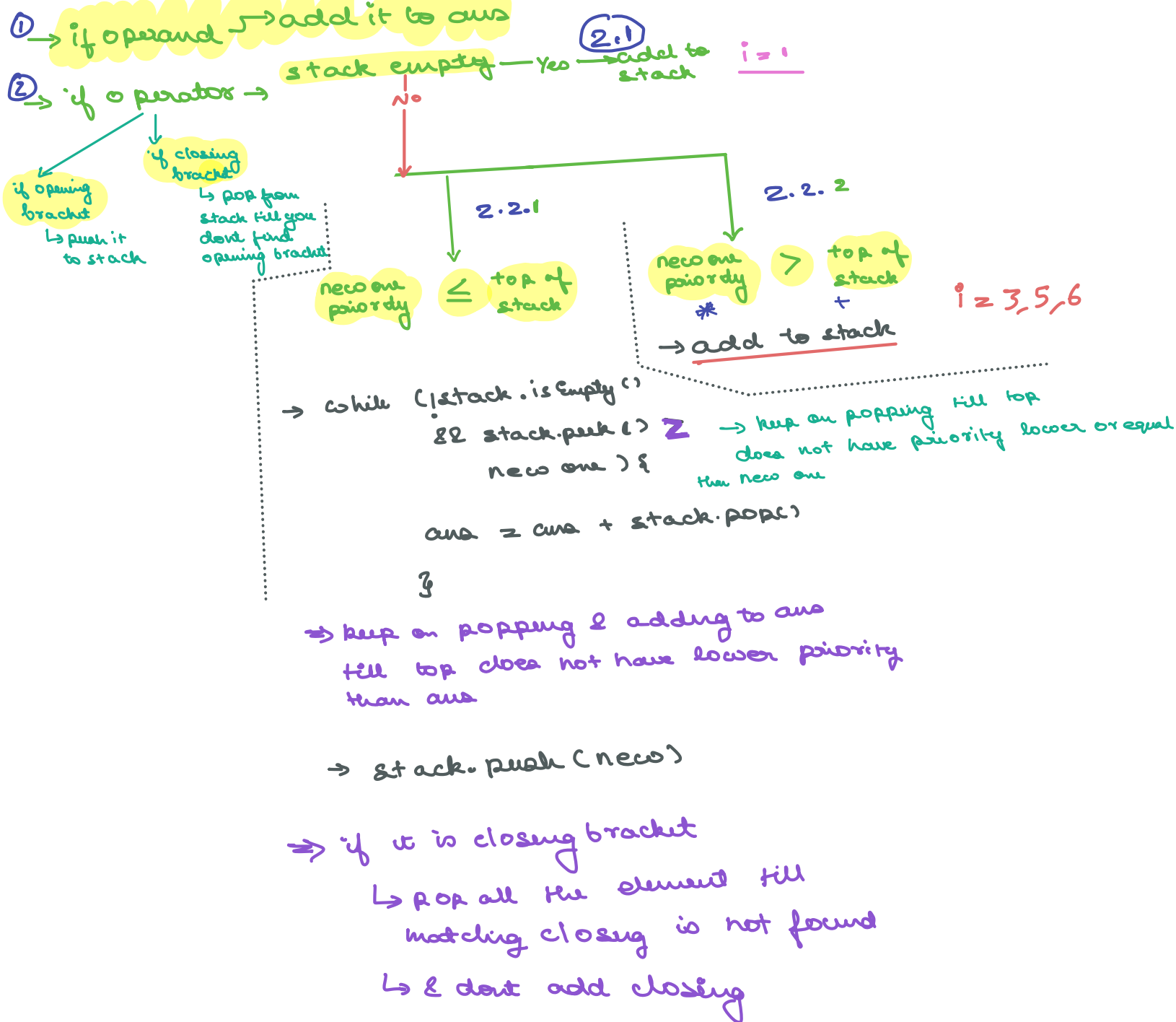
→ $a + b * (c^d - e)$

i	expression	stack = $\boxed{}_{\text{top}}$	ans = ""
0	a + b * (c^d - e)		a
1	a + b * (c^d - e)	+	a
2	a + b * (c^d - e)		ab
3	a + b * (c^d - e)	+ *	ab
4	a + b * (c^d - e)	+ * (	abc
5	a + b * (c^d - e)	+ * (^	abcd
6	a + b * (c^d - e)	+ * (^	abcd
7	a + b * (c^d - e)	+ * (^	abcd
8	a + b * (c^d - e)	+ * (^ pop → + + (-	abcd^
9	a + b * (c^d - e)	+ * (-	abcd^e
10	a + b * (c^d - e)	+ * (^ pop → don't push closing	abcd^e -
		+ *	abcd^e - * +

stack empty →

new one large pow

new one less pow



3 once iteration over expression is over
add all from stack to expression

{ Convert Infix to Postfix }

infixToPostfix (string str) {

int i=0; Stack stack; String ans

while (i < n) { $\rightarrow O(n)$

if (s[i] >= 'A' && s[i] <= 'Z' ||
s[i] >= 'a' && s[i] <= 'z' ||
s[i] >= '0' && s[i] <= '9') {

// operator

ans = ans + s[i]

} else if (s[i] == '(') {

// add it to stack if it is opening
stack.push(s[i])

} else if (s[i] == ')') {

// if it is a closing bracket then
// keep on removing till opening not found

while (!stack.isEmpty() && $\rightarrow O(n)$
stack.peek() != '(') {
more

ans = ans + stack.pop()

}

if (!stack.isEmpty() &&
stack.peek() == '(') {

stack.pop()

}

} else {

// operator

```
while (!stack.isEmpty()) &&
```

```
priority(s[i]) <= priority(stack.top());
```

```
// priority of new is smaller than stack.top  
// so we can't push as stack.top has to be  
// highest priority
```

```
ans = ans + stack.pop();
```

```
}  
stack.push(s[i]);
```

```
}  
i++
```

```
}
```

```
while (!stack.isEmpty()) {
```

```
ans = ans + stack.pop();
```

```
}
```

T.C $\Rightarrow O(n) + O(n)$

Space $\Rightarrow O(n) + O(n)$

priority(char s)

s == ^ $\rightarrow 3$

s == * || / $\rightarrow 2$

s == - + $\rightarrow 1$

default $\rightarrow -1$

Infix

stack

str
abc

$$A + \underbrace{B * C}_{BC*}$$

$$\begin{array}{c} * \\ + \\ \hline \end{array} \rightarrow abc*+$$

$$10 + \underbrace{3 \times 4}_{34*} - 7$$

$$10 \ 3 \ 4 \ * \ + \ 7 \ -$$

$$\begin{array}{l} \underline{10 /} \\ 10 / (4 - 2) * 6 + 9 \end{array}$$

high $\rightarrow 1042- / \rightarrow 1042- / 6*$
same pow $\rightarrow 9+$

$$42 -$$

$$10 / 2 * 6 + 9$$

$$5 * 6 + 9$$

$$30 + 9$$

$$39 \checkmark$$

Postfix

$$A \ B \ C \ * \ +$$

$$\begin{array}{c} \text{same power } 20 + \text{ will be addressed first} \\ \downarrow \quad \downarrow \\ 10 \ 3 \ 4 \ * \ + \ 7 \ - \end{array}$$

no 2 same pow can come together

$$\begin{array}{c} 2 \\ \downarrow \downarrow \\ 10 \ 4 \ 2 \ - \ / \ 6 \ * \ 9 \ + \end{array}$$

$$\boxed{39}$$

$$10 / 2$$

$$5$$

$$6 * 5$$

str

$$10 \ 4 \ 2 \ - \ /$$

stack

$$\begin{array}{c} * \\ * \\ / \end{array}$$

Q Infix to Prefix striver

eg $\Rightarrow (A+B)*C - D + F$



① $\rightarrow$ reverse string

$F + D - C *) B + A ($

$\rightarrow$ scan thru - & make opening as closing & closing as opening

$F + D - C * (B + A)$

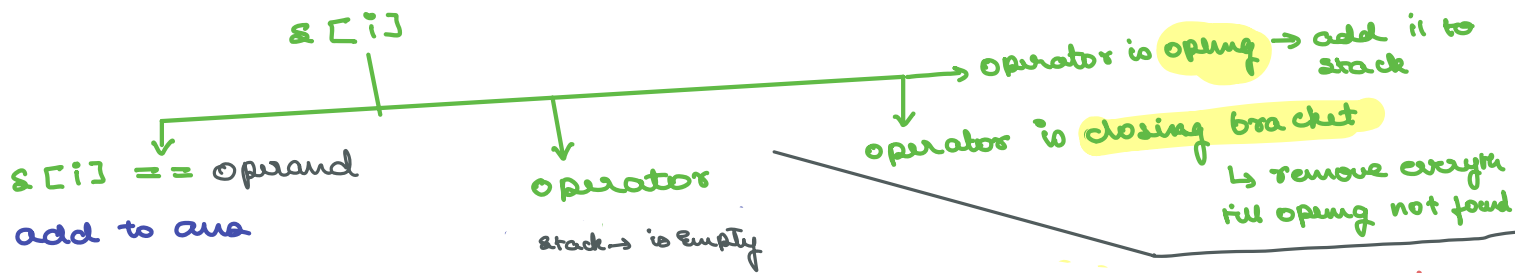
3 steps

$\rightarrow$ ① Reverse the string

$\rightarrow$ ② Convert infix to postfix with a conditional change

$\rightarrow$ ③ Reverse the ans again

② convert to postfix with a catch



$\rightarrow$ in previous

conversion we were removing element on $\leq$ here only in case of greater we remove

$\text{priority}(s[i]) \leq \text{priority}(\text{stack.top()})$

③ after looping string

$\rightarrow$ reverse ans again

$\rightarrow$ add all the stack value previous to ans

0 1 2 3 4 5 6 7 8 9 10
 F + D - C * (B + A)

	stack	ans
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A)		F
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) +		F
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) +		F D
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + -		F D
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + -		F D C
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + - *		F D C
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + - * (		F D C B
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + - * (+		F D C B A
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + - * (+		F D C B A +
0 1 2 3 4 5 6 7 8 9 10 F + D - C * (B + A) + - *		F D C B A + * - +
empty the stack →		F D C B A + * - +
see the ans →		+ - * + A B C D F

fn convert Infix to Prefix (String Input) {

String rev = reverse (Input);

String inp = convertOpeningToClosing (rev);

i = 0 n = inp.length

while (i < n) {

char val = inp.charAt(i);

if (val ==> operand) {

| ans = ans + val → operand so
| add to ans

|

} if (val == '(') { → opening

| stack.push (val)

} if (val == ')') {

|

while (!stack.isEmpty() &

|

stack.peek() != '(') {

|

|

ans = ans + stack.pop();

|

|

|

if (!stack.isEmpty() &) {

|

stack.pop();

|

|

|

} if (val == '+', '-', '^', '*', '/') {
// Operator

if (val == '^') {

while (!stack.isEmpty() &

&

priority <= priority
(stack.top())

) {

ans += stack.pop();

}

```

    }
    while ( ! stack.isEmpty() )
    {
        priority (s.top()) < priority
        ) {
        ans += s.top()
        }
        s.push ( val )
    }

```

```

    }
    i++
    }

```

```

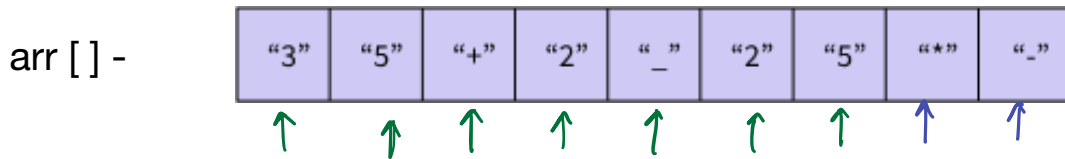
while ( ! st.isEmpty() ) {
    ans += st.pop()
}
ans = rev(ans)
}

```

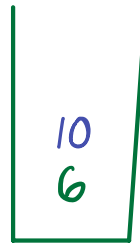
Todo : post fin to infen



< **Question** > : Evaluate given valid post fix expression.

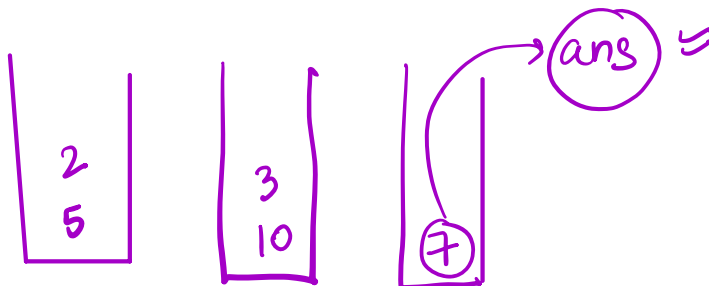


The operator is always present after two operands.



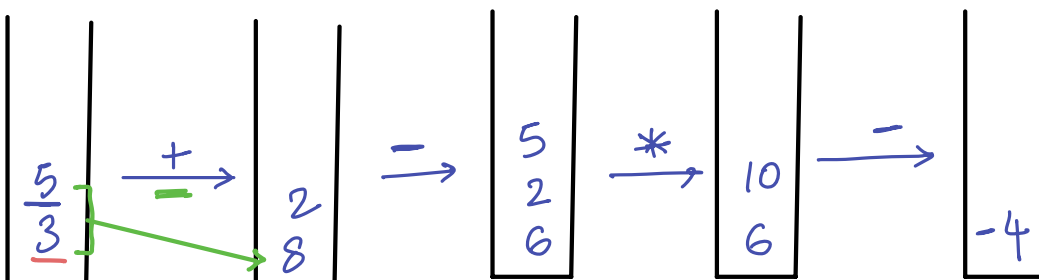
$$\begin{matrix} \text{op1} & & \text{op2} \\ (6) & - & (10) = (-4) \end{matrix}$$

Quiz : 5, 2, *, 3, -



Quiz

3 5 + 2 - 2 5 * -





</> Code

+, -, *, /

Traverse the string

if the char is +, -, *, / // operator.

op2 = st.top()

st.pop()

op1 = st.top()

st.pop()

if (char is +) {

| st.push (op1 + op2)

}

else if (char is -) {

| st.push (op1 - op2)

}

else if (char is *) {

| st.push (op1 * op2)

}

else {

| st.push (op1 / op2)

}

} else { // in case of operand

st.push (the number)

}

{
return st.top();TC $\rightarrow O(N)$ SC $\rightarrow O(N)$

